

Labwork 7: Reduction

Pham Tien Nhat

October 4, 2026

1 Implementation

I implemented the grayscale stretch with 3 steps: convert the image to gray, find the max and min intensity, and recalculate every pixel from $[min, max]$ to $[0, 255]$.

The input image already uses the range $[0, 255]$, so it is squeezed $[60, 160]$.

```
import math
import time
import numpy as np
import matplotlib.pyplot as plt

image = plt.imread('1.jpeg')
image = (image * (100 / 255) + 60).astype(np.uint8)
plt.imshow(image)
h, w, d = image.shape
```



Figure 1: Input image, squeezed to $[60, 160]$.

Grayscale: Each thread converts one pixel to gray with the same 2D kernel taken from previous labworks.

```
@cuda.jit
def grayscale(src, dst):
    tidx = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    tidy = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if tidx >= src.shape[1] or tidy >= src.shape[0]:
        return
    dst[tidy, tidx] = np.uint8((src[tidy, tidx, 0] + src[tidy, tidx, 1]
                                + src[tidy, tidx, 2]) / 3)

blockSize = (32, 32)
```

```

gridSize = (math.ceil(w / blockSize[0]), math.ceil(h / blockSize[1]))
devSrc = cuda.to_device(image)
devGray = cuda.device_array((h, w), np.uint8)
grayscale[gridSize, blockSize](devSrc, devGray)
gray = devGray.copy_to_host()

```



Figure 2: Gray image, input of step 2 and step 3.

Find max/min: The kernel follows the `sum1D` kernel from the slide.

```

sharedSize = 256

@cuda.jit
def reduceMax(src, dst):
    cache = cuda.shared.array((sharedSize, ), np.uint8)
    localtid = cuda.threadIdx.x
    tid = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    if tid < src.shape[0]:
        cache[localtid] = src[tid]
    else:
        cache[localtid] = 0
    cuda.syncthreads()
    s = 1
    while s < cuda.blockDim.x:
        if localtid % (s * 2) == 0:
            cache[localtid] = max(cache[localtid], cache[localtid + s])
            cuda.syncthreads()
        s = s * 2
    if localtid == 0:
        dst[cuda.blockIdx.x] = cache[0]

```

`reduceMin` is the same kernel with `min` and a padding value of 255.

```

devData = devGray.reshape(h * w)
gridReduce = math.ceil(h * w / sharedSize)
devMax = cuda.device_array(gridReduce, np.uint8)
devMin = cuda.device_array(gridReduce, np.uint8)
reduceMax[gridReduce, sharedSize](devData, devMax)
reduceMin[gridReduce, sharedSize](devData, devMin)

gmax = devMax.copy_to_host().max()
gmin = devMin.copy_to_host().min()

```

I get the values: $max = 160$ and $min = 60$.

Stretch: Each pixel is recalculated with $g' = \frac{g-min}{max-min} \times 255$.

```
@cuda.jit
def stretch(src, dst, gmin, gmax):
    tidx = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    tidy = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if tidx >= src.shape[1] or tidy >= src.shape[0]:
        return
    dst[tidy, tidx] = np.uint8((src[tidy, tidx] - gmin)
                                / (gmax - gmin) * 255)

devStretch = cuda.device_array((h, w), np.uint8)
stretch[gridSize, blockSize](devGray, devStretch,
                             np.float32(gmin), np.float32(gmax))
stretched = devStretch.copy_to_host()
```

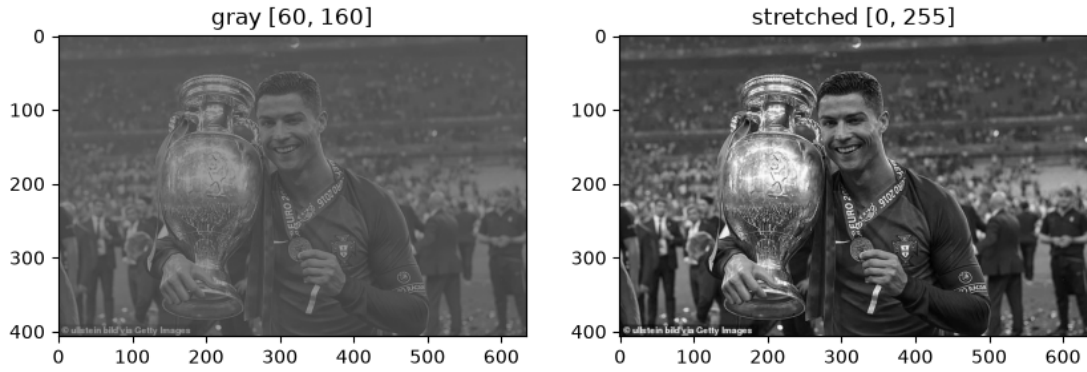


Figure 3: Gray image [60, 160] (left) and stretched image [0, 255] (right).

Figure 4 shows the histogram: the gray values are packed in [60, 160] before the stretch, and scatter over [0, 255], showing cleaner contrast.

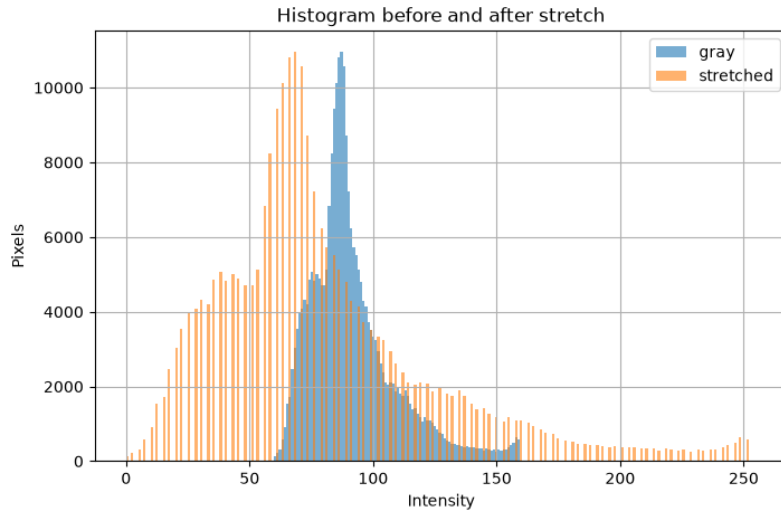


Figure 4: Histogram before and after the stretch.

2 Experimental Results

4 2D block sizes are tested for the two map kernels: (4, 4), (8, 8), (16, 16) and (32, 32).

```

block_sizes = [(4, 4), (8, 8), (16, 16), (32, 32)]
times_gray = []
times_stretch = []

for blockSize in block_sizes:
    gridSize = (math.ceil(w / blockSize[0]), math.ceil(h / blockSize[1]))

    start = time.time()
    grayscale[gridSize, blockSize](devSrc, devGray)
    cuda.synchronize()
    times_gray.append(time.time() - start)

    start = time.time()
    stretch[gridSize, blockSize](devGray, devStretch,
                                  np.float32(gmin), np.float32(gmax))
    cuda.synchronize()
    times_stretch.append(time.time() - start)

plt.figure(figsize=(8, 5))
plt.plot([str(b) for b in block_sizes], times_gray, marker='o',
         label='grayscale')
plt.plot([str(b) for b in block_sizes], times_stretch, marker='o',
         label='stretch')
plt.xlabel("2D Block Size")
plt.ylabel("Time (seconds)")
plt.title("CUDA 2D Block Size vs GPU Execution Time")
plt.legend()
plt.grid(True)
plt.show()

```

Figure 5 shows the relationship between the 2D block size and the GPU time.

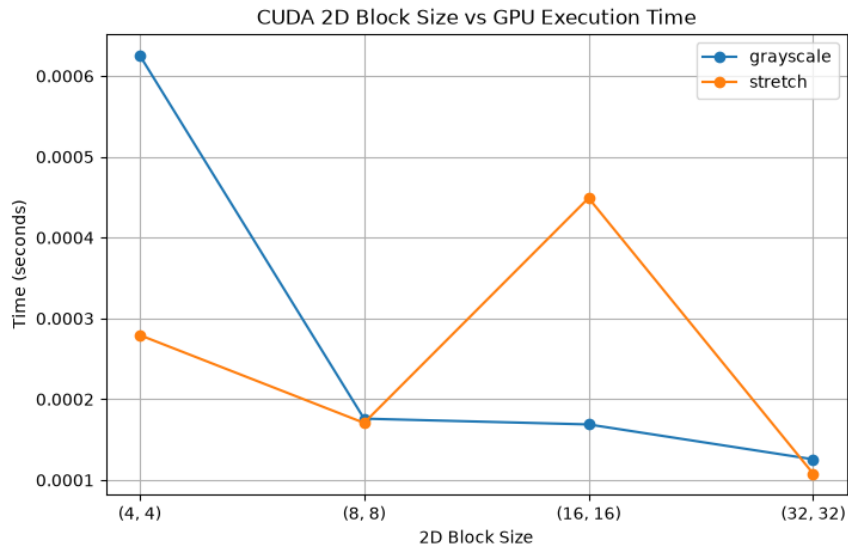


Figure 5: 2D block size vs GPU time.

The block of (4, 4) is the slowest for both kernels. (32, 32) is the fastest where the peak of stretch comes at (16, 16).